BED

LECTURE

02

ngee ann school of infocomm technology

Introduction to API

Backend Development

Diploma in IT

Year 2 (2026/27), Semester 1

Objectives

At the end of this, you will …..

- Define what is an API and a REST API
- Explain why APIs are important
- Describe the purpose of HTTP request and response
- Identify the components of HTTP request and Response

Topics

- What is an API?
- Advantages of using API
- What is REST API
- How REST APIs work
- Examples of Node.js program

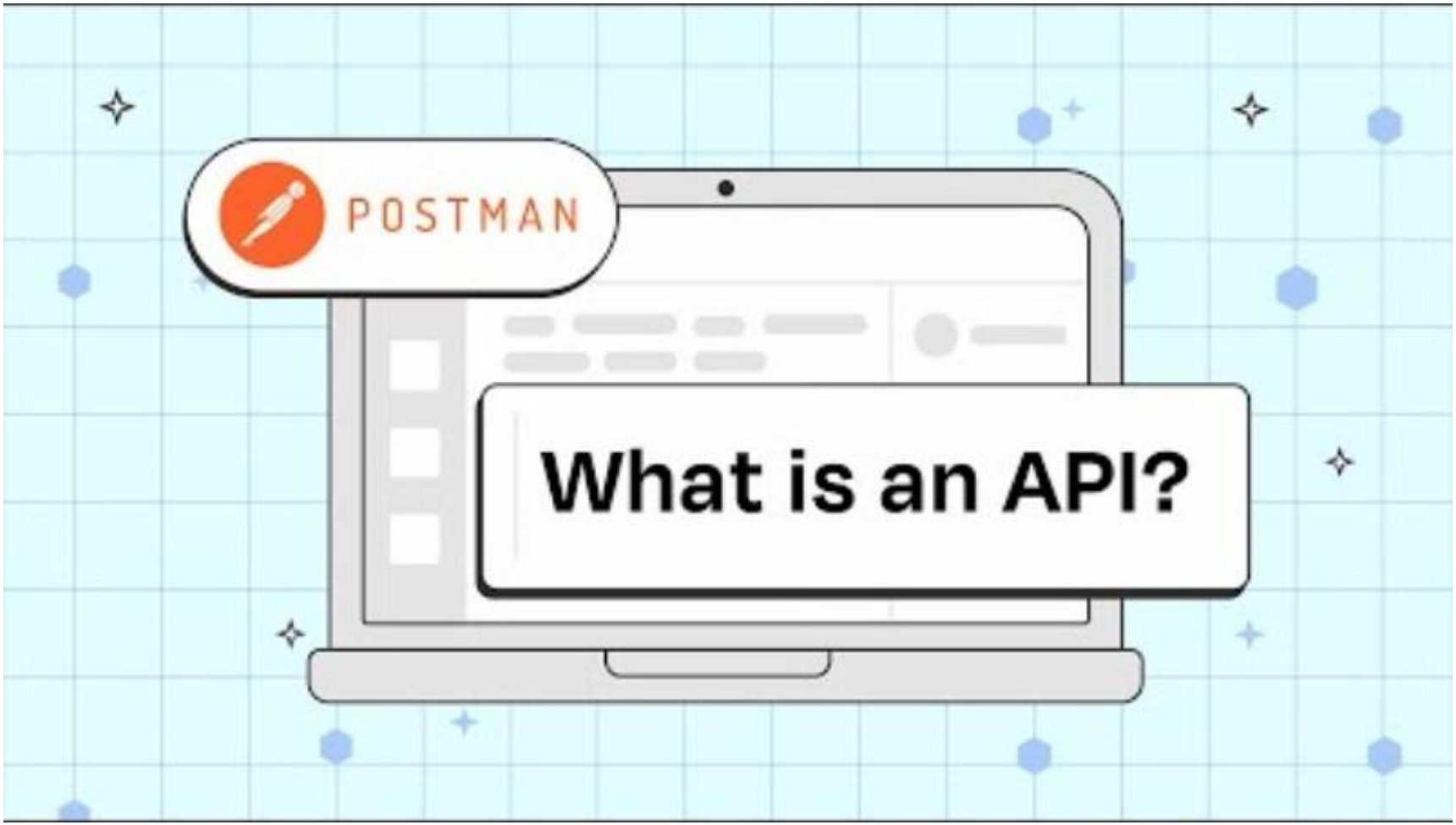
Reading Reference

- What is an API
  https://www.postman.com/what-is-an-api
- Node.js - RESTful API
  https://www.tutorialspoint.com/nodejs/nodejs_restful_api.htm
- Creating a REST API with Node.js and Express: A step-by-step guide
  https://levelup.gitconnected.com/creating-a-rest-api-with-node-js-and-express-a-step-by-step-guide-3bd7188fa219
- How do I use Node.js to create web APIs
  https://reintech.io/blog/how-to-use-node-js-to-create-web-apis

What is an API?

- API (Application Programming Interface)
  - A set of protocols or functions that allow software components to communicate and share data.
  - APIs are everywhere – powering apps, websites and devices.
  - Essential for software interaction.
    - Simplify the connection between different systems.
  - Enhance the functionality of applications.
  - Expose data and services for various uses.

What is an API?



What is an API? #api #postman - YouTube

Source: https://www.postman.com/what-is-an-api

Advantages of Using APIs

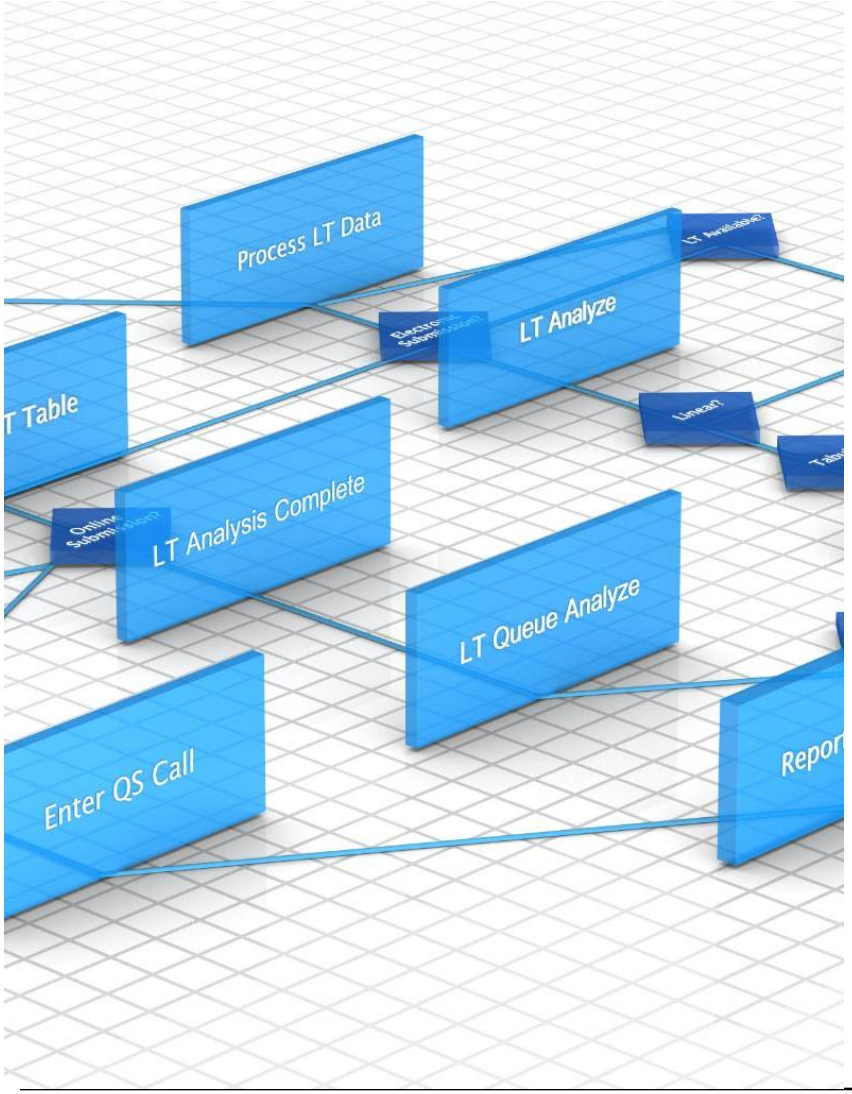
🚀 **Connect Systems Easily**
APIs let different apps talk to each other — faster and smarter.

♻️ **Reuse, Don't Rebuild**
Use existing features across multiple apps and save time.

🎯 **Create Seamless Experiences**
Connect users to services smoothly, with better interactions.

📈 **Scale Without Limits**
Grow your app as demand rises — APIs make scaling easy.

What is a REST API?



- Based on REST (Representational State Transfer) architectural principles
- Uses standard HTTP methods (GET, POST, PUT, DELETE) for communication
- Ensures scalability, simplicity, and ease of use for developers
- Focuses on statelessness, Each request contains all necessary information

How REST APIs work

- REST APIs communicate through HTTP requests to perform standard database functions like creating, reading, updating and deleting records (also known as **CRUD**).
- An HTTP method tells the server what it needs to do to the resource. The following are four common HTTP methods:
  - GET
    - **R**ead records
  - POST
    - **C**reate record
  - PUT
    - **U**pdate record
  - DELETE
    - **D**elete record

HTTP Request

- HTTP requests consist of request lines, headers, and bodies.
- The request line specifies the method and resource URL.
- Headers provide additional context, such as content type and authorization.
- The request body contains data sent to the server, if applicable.

Reference 5
Slide 10

HTTP Request

Sample Client Requests:

- **A typical client GET request:**

```
GET /students HTTP/1.1
User-Agent: Chrome
Accept: application/json
[CRLF]
```

Request Line
Request Headers
(no request body)

- **A typical client POST request:**

```
POST /students/save HTTP/1.1
User-Agent: IE
Content-Type: application/x-www-form-urlencoded
[CRLF]
name=Alex&id=S12345678
```

Request Line
Request Headers
Request Body (e.g. form parameters)

HTTP Request

Sample Client Requests:

- **A typical client PUT request:**

```
PUT /students/S12345678 HTTP/1.1
User-Agent: Chrome
Accept: application/json
[CRLF]
{"name": "Alex"}
```

Request Line (Resource URL with an identifier)

Request Headers

Request Body (e.g. JSON object)

- **A typical client DELETE request:**

```
DELETE /students/S12345678 HTTP/1.1
User-Agent: IE
Accept: application/json
[CRLF]
```

Request Line (Resource URL with an identifier)

Request Headers

(no request body)

HTTP Response

- An HTTP response is sent back by the server.
- It contains a status code indicating the outcome.
- Response includes headers with metadata about the request.
- The body holds the actual content returned to the client.
- Common status codes include 200, 404, and 500.

HTTP Response

Sample Server Responses:

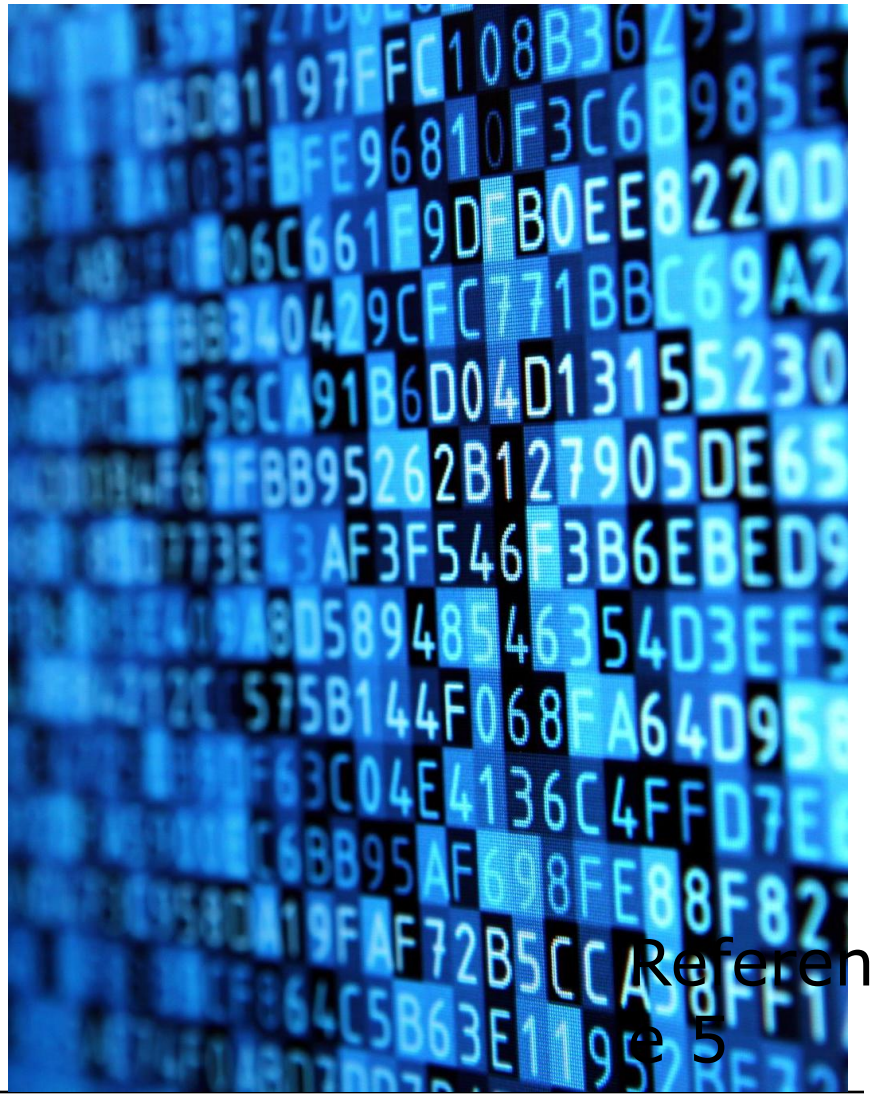
```
HTTP/1.1 200 OK                    } Response Status
Content-Type: text/html            } Response Headers
Content-Length: 1337
[CRLF]
<html>
  <!-- Some HTML Content. -->      } Response Body (content)
</html>
```

```
HTTP/1.1 500 Internal Server Error } Response Status
```

```
HTTP/1.1 201 Created               } Response Status
Location: /view/7                  } Response Header
[CRLF]
Some message goes here.            } Response Body
```

HTTP Response Status Codes

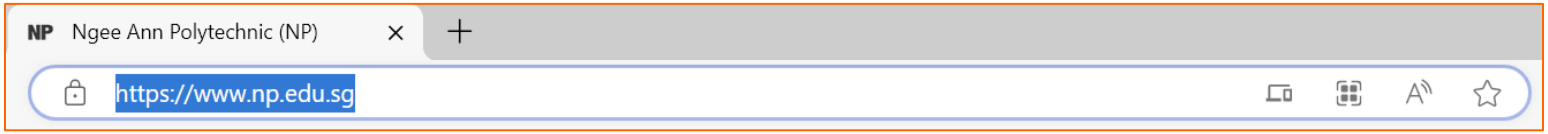
- **Response Status Code** – Code in the response indicating the status of the request.
  - 2xx Success
    - 200 OK
    - 204 No content
  - 3xx Redirect
    - 304 Not Modified
  - 4xx Client Error
    - 400 Bad Request
    - 401 Unauthorized
    - 403 Forbidden
    - 404 Not Found
  - 5xx Server Error
    - 500 Internal Server Error



Reference 5

RECALL : Front End Development

- How do we send request with various HTTP methods from front end (client or browser)?
  - Input a URL into the browser window or clicking a HTML hyperlink <a> send request with **GET** method.




```
<p><a href="https://www.np.edu.sg">Visit NP</a></p>
```

  - HTML form can send request with **GET** or **POST** method.

```
<form action="/action_page.php" method="get">
  <label for="fname">First name:</label>
  <input type="text" id="fname" name="fname"><br><br>
  <label for="lname">Last name:</label>
  <input type="text" id="lname" name="lname"><br><br>
  <input type="submit" value="Submit">
</form>
```

RECALL : Front End Development

- How about PUT & DELETE methods?
  - Using JavaScript can send request with all HTTP methods.

```
fetch(url, {
  method: 'POST', // GET, POST, PUT, DELETE etc
  body: JSON.stringify({
  name: 'red',
  login: 'redmonster',
  })
})
.then(function (data) {
  //return data.json();
//do something
})
.then(function (data) {
  //process the return earlier
}).
.catch(function (error){
//handle error
})
```

Examples of Node.js program

- These are Node.js code to handle request with various HTTP methods:

```
// GET [ALL Books]
app.get("/books", (req, res) => {
  res.send("Fetching a list of all available books");
});


// GET [Single Book by ID]
app.get("/books/:id", (req, res) => {
  const bookId = parseInt(req.params.id);
  res.send(`Fetching details for book with id: ${bookId}`);
});
```

Examples of Node.js program

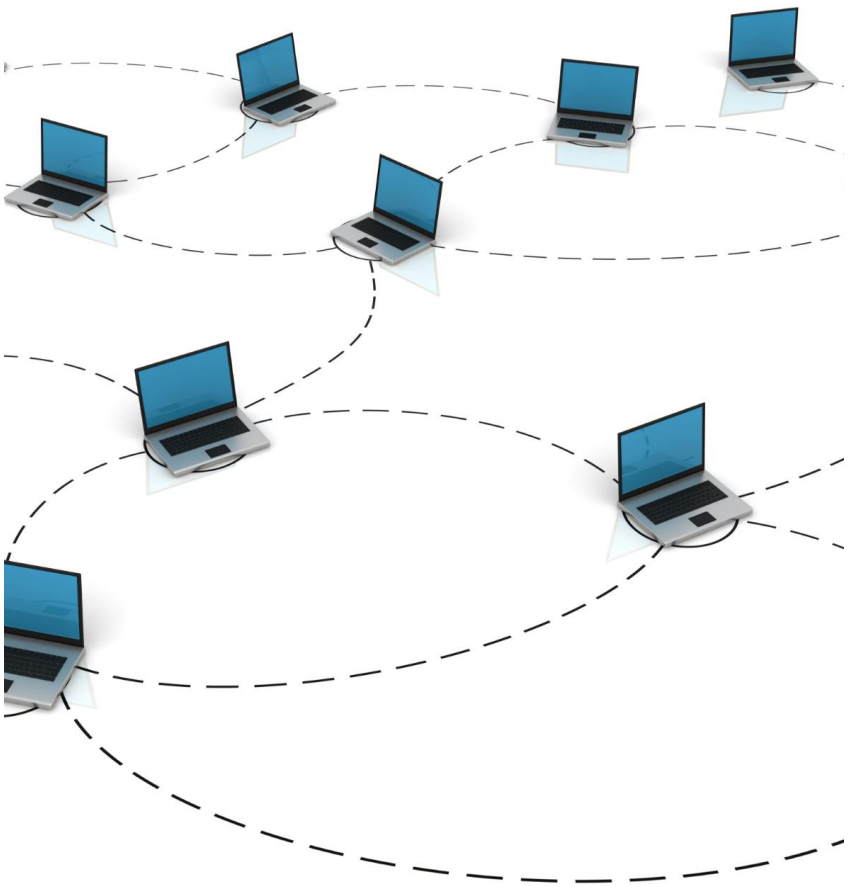
```
// POST [Adding a New Book]
app.post("/books", (req, res) => {
  const newBook = req.body; // Access data from request body
  res.status(201).json({
    message: "Received request to add a new book.",
    receivedData: newBook
  });
});

// PUT [Updating an Existing Book]
app.put("/books/:id", (req, res) => {
  const bookId = parseInt(req.params.id);
  const updatedBook = req.body;
  res.json({
    message: `Received request to update the book with ID: ${bookId}`,
    receivedData: updatedBook
  });
});
```

Examples of Node.js program

```
// DELETE [Deleting a Book]
app.delete("/books/:id", (req, res) => {
  const bookId = parseInt(req.params.id);
  res.json({
    message: `Received request to delete the book with ID: ${bookId}`
  });
});
```

Summary of API and REST Concepts



- APIs enable communication between software applications.
- REST APIs follow specific architectural principles for flexibility.
- HTTP methods are used for CRUD operations in APIs.
- HTTP requests initiate communication between client and server.
- Responses from the server provide status and data back to the client.